AI + EXCEL VBA

COURSE SLIDES

AI Vibe Coding for Excel VBA



Course Code: C357

Conducted by Tertiary Infotech Academy Pte Ltd
UEN 201200696W

Version v1.0 · 28 September 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved.

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE
Principal Trainer, Tertiary Infotech Academy Pte Ltd

CERTIFICATION
Microsoft Excel, VBA automation, data analytics and software engineering.

DELIVERS
Courses on Excel automation, AI, data science and software development.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

- Your name and organisation / role.
- How much Excel you use day to day — formulas, pivot tables, macros?
- Any programming experience at all (any language)?
- One repetitive Excel task you would love to automate by the end of this course.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. Attend both days in full.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL · https://lms-tms.tertiaryinfotech.com

Download Your Course Material

1. Open the LMS portal in your browser
2. Log in with the email you registered with
3. Open My Courses and select this course
4. Download the Slides, Learner Guide and Lab files
5. Keep them open as you work through the labs

© 2026 Tertiary Infotech Academy Pte Ltd

LAB MATERIALS · MOCK EXCEL FILES

Your Lab Files — One Folder per Lab

1

lab-NN-start.xlsm

OPEN THIS ONE. The workbook and its data exactly as they stand before the lab begins.

2

lab-NN-end.xlsm

The finished workbook for this lab — check your own sheets and results against it.

3

vba/

The worked solution as importable VBA: .bas modules, .cls event code, .frm UserForm.

4

data/

The mock CSV datasets — realistic NorthWind figures that exercise every commission band.

5

Each start = the previous end

All 31 labs build ONE workbook, so you can rejoin the course at any lab.

6

Always save as .xlsm

Saving as .xlsx silently discards every macro, form and button assignment.

© 2026 Tertiary Infotech Academy Pte Ltd

EVERY LAB FOLLOWS THE SAME FIVE STEPS

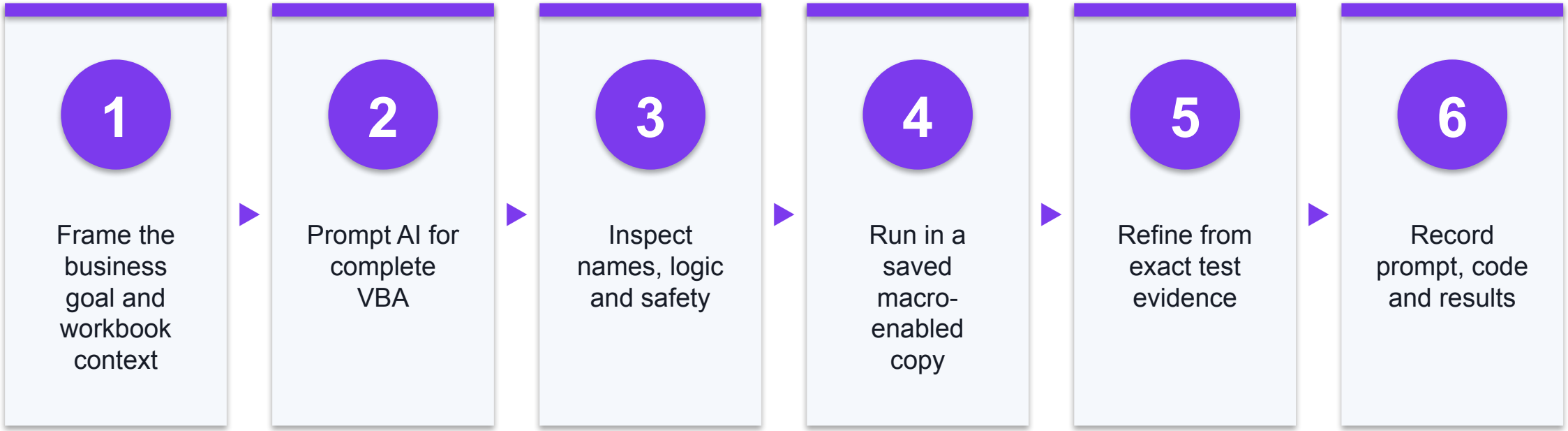
How to Use the Lab Files

1. Open lab-NN-start.xlsm
2. Write the lab's code yourself — or import vba/ to see it working
3. Follow the lab's step-by-step
4. Compare with lab-NN-end.xlsm
5. Save your work as .xlsm

© 2026 Tertiary Infotech Academy Pte Ltd

SUPERVISED PROMPT-TO-CODE · LEARNER REMAINS ACCOUNTABLE

AI Vibe Coding Workflow



© 2026 Tertiary Infotech Academy Pte Ltd

Review AI-Generated VBA Before Running

1

Correct placement

Module1, ThisWorkbook, the named worksheet or frmSalesEntry — exactly where the procedure belongs.

2

Exact object names

Every worksheet, range, control and procedure name must exist in the supplied workbook.

3

Business-rule boundaries

Test thresholds, blank values, zero, text input and the known acceptance result.

4

No hidden side effects

Reject unexpected file deletion, shell execution, network calls, credential access or hidden code.

5

Readable and maintainable

Require Option Explicit, typed variables, focused procedures, comments and safe error handling.

6

Evidence retained

Save the prompt, generated draft, learner edits, test inputs, actual results and final .xlsm.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE · 9:30am–5:30pm, 30-min lunch

Lesson Plan — 2 Days, 7.5 hours per day

Day 1

- Day 1 — AI-assisted VBA foundations — prompt, generate, inspect and build reusable procedures

– Welcome · Introductions · Outcomes

– Topic 1: Get Started on VBA (Labs 1–6)

– Lunch Break

– Topic 2: VBA Programming (Labs 7–13)

Day 2

- Day 2 — AI-assisted Excel automation — objects, testing, debugging and user-defined forms

– Day 1 Recap

– Topic 3: Function & Sub Procedure (Labs 14–18)

– Topic 4: Managing Excel Objects (Labs 19–25)

– Lunch Break

– Topic 5: User Defined Forms (Labs 26–31)

– Course Recap · Capstone Review · Feedback

© 2026 Tertiary Infotech Academy Pte Ltd

FIVE TOPICS · 31 HANDS-ON LABS

Course Outline

1

Topic 1 Get Started on VBA

What VBA is · the VBA Editor · modules & procedures · message and input boxes · recording macros.

2

Topic 2 VBA Programming

Variables & constants · data types & arrays · operators · decisions · loops.

3

Topic 3 Function and Sub Procedure

Functions vs subs · arguments & returns · scope · worksheet functions · charts.

4

Topic 4 Managing Excel Objects

Application, Workbook, Worksheet & Range · error types · error handling · debugging.

5

Topic 5 Create User Defined Forms

Workbook & worksheet events · UserForms & controls · validation · documentation.

© 2026 Tertiary Infotech Academy Pte Ltd

THE RUNNING CASE STUDY

One workbook. Thirty-one labs.

Every lab in this course builds one realistic deliverable: an automated commission tracker for the sales team of NorthWind Trading, a Singapore distributor. You start by framing a task for an AI coding assistant, and finish with a reviewed and tested UserForm solution that captures a sale, calculates banded commission, handles bad input and plots the result. Every lab retains its prompt, generated VBA and test evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

CORE CONCEPTS

Why Automate Excel?

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 1 PREVIEW

What is VBA?

1

Visual Basic for Applications

An event-driven programming language built into Excel, Word, PowerPoint and Access.

2

Already installed

The VBA Editor ships with every desktop version of Office — nothing to buy or download.

3

Records and replays

The macro recorder turns your clicks into real code you can read, edit and reuse.

4

Drives Excel itself

Anything you can do by hand — formulas, formatting, charts, files — VBA can do from code.

© 2026 Tertiary Infotech Academy Pte Ltd

Manual vs Automated

By hand, every month

- Open six regional files and copy-paste each one.
- Retype the commission rate into every sheet.
- Recalculate bands manually and hope nothing slipped.
- Rebuild the chart from scratch.
- About two hours — and a different result each analyst.

With a VBA macro

- Click one button on the worksheet.
- The rate is read once from a single cell.
- Banded commission is applied identically to every row.
- The chart redraws itself from the new figures.
- Under a second — and the same result every time.

© 2026 Tertiary Infotech Academy Pte Ltd

WHERE THE CODE LIVES

The VBA Editor — Your Workbench

1

Open it

Alt+F11 on Windows, Opt+F11 on Mac, or Developer → Visual Basic on the ribbon.

2

Project Explorer

Top-left: every open workbook with its sheets, ThisWorkbook, modules and forms.

3

Properties window

Bottom-left: the settings of whatever object or control you have selected.

4

Code window

The large right-hand pane where you write and read your procedures.

5

Immediate window

Ctrl+G — run one line at a time and print values while debugging.

6

Modules

Insert → Module creates the container your macros live in.

© 2026 Tertiary Infotech Academy Pte Ltd

ANATOMY OF A PROCEDURE

Your First Macro

```
Sub Welcome()
    ' Greets the analyst when the commission tool opens
    MsgBox "Welcome to the NorthWind Commission Tracker"
End Sub
```

Sub … End Sub wraps every macro · the apostrophe marks a comment · MsgBox shows a dialog.

© 2026 Tertiary Infotech Academy Pte Ltd

FILE FORMATS

Saving Your Work — .xlsm

1

Macro-enabled workbook

A workbook containing VBA MUST be saved as .xlsm (Excel Macro-Enabled Workbook).

2

Save it as .xlsx

Excel silently strips out every macro — your code is gone.

3

Enable Content

On reopening, click 'Enable Content' on the yellow security bar to allow macros to run.

4

Remember this

The correct extension for a macro-enabled Excel file is .xlsm — check it at every save.

© 2026 Tertiary Infotech Academy Pte Ltd

01

TOPIC 01

Get Started on VBA

What VBA is · The Visual Basic Editor · Modules & procedures · Message and input boxes · Recording macros

© 2026 Tertiary Infotech Academy Pte Ltd

LO1 · CONCEPTS 1–5

Key Concepts — Get Started on VBA

1

AI vibe coding

Turn a precise business intent into draft VBA with an AI assistant, then review, test and refine it before use.

2

What is VBA?

Visual Basic for Applications — the event-driven language built into Excel that turns manual, repeated spreadsheet work into a one-click macro.

3

Why automate?

A monthly report that takes an analyst two hours by hand runs in seconds, identically, every time — no copy-paste slips.

4

The Visual Basic Editor

Press Alt+F11 (Windows) or Opt+F11 (Mac), or enable the Developer ribbon, to open the editor where all code lives.

5

Modules and procedures

Code lives in a Module; a Module holds Procedures — Sub procedures that do things and Functions that return values.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Get Started on VBA

1

Talking to the user

MsgBox displays a message and captures a button choice; InputBox prompts the user to type a value.

2

The macro recorder

Record your clicks into working VBA, then read and edit the generated code — the fastest way to learn Excel's object model.

3

Macro-enabled files

A workbook containing macros must be saved as .xlsm — save it as .xlsx and every macro is silently discarded.

4

Running a macro

Run from the editor (F5), from View → Macros, or by assigning the macro to a button or shape on the worksheet.

© 2026 Tertiary Infotech Academy Pte Ltd

Message Box and Input Box

```
Sub ConfirmRecalculate()
    Dim intAnswer As Integer
    intAnswer = MsgBox("Recalculate all commissions?", vbYesNoCancel, "Confirm")
    If intAnswer = vbYes Then
        MsgBox "Recalculating..."
    End If
End Sub
```

MsgBox(prompt, buttons, title) returns which button was clicked — vbYesNoCancel shows Yes, No and Cancel.

© 2026 Tertiary Infotech Academy Pte Ltd

KNOW THESE

MsgBox Button Constants

1

vbOKOnly (0)
Displays the OK button only — a simple notification.

2

vbOKCancel (1)
Displays OK and Cancel — proceed or stop.

3

vbAbortRetryIgnore (2)
Displays Abort, Retry and Ignore — classic error recovery.

4

vbYesNoCancel (3)
Displays Yes, No and Cancel.

5

vbYesNo (4)
Displays Yes and No — a forced binary choice.

6

vbRetryCancel (5)
Displays Retry and Cancel.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · 6 LABS

Hands-On Labs — Get Started on VBA

1

Lab 1: Set Up the Developer Ribbon and Save a Macro-Enabled Workbook

A macro-enabled workbook, NorthWind-Commission.xlsm, with the Developer ribbon enabled and the VBA Editor open.

2

Lab 2: Write Your First Macro in a Module

A Module1 containing a Welcome Sub procedure that greets the user with a message box.

3

Lab 3: Message Boxes and Button Choices

A ConfirmRecalculate Sub that asks the user to proceed and reports the button they chose.

4

Lab 4: Capture User Input with an Input Box

A ShowCommission Sub that asks for a commission rate and reports the commission on the selected cell.

5

Lab 5: Record a Macro to Calculate Sales Tax

A recorded Cal_SalesTax macro plus your annotated reading of the generated code.

6

Lab 6: Run a Macro from a Button and a Shape

A Sales sheet with a 'Calculate Commission' button and a shape that both launch macros.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON · LAB 1

Set Up the Developer Ribbon and Save a Macro-Enabled Workbook

You have just joined NorthWind Trading as a sales analyst. Before you can automate anything, you must switch on Excel's Developer tools, open the Visual Basic Editor, and learn why a workbook containing macros has to be saved as .xlsm.

YOU'LL BUILD

A macro-enabled workbook, NorthWind-Commission.xlsm, with the Developer ribbon enabled and the VBA Editor open.

✓ TEST IT

The Developer tab is visible, Alt+F11 opens the Visual Basic Editor, the Sales sheet holds 12 data rows, and the title bar reads NorthWind-Commission.xlsm.

TOOLS: Excel, Developer ribbon, Visual Basic Editor

DATA: Mock data: 12 NorthWind sales records for January (rep, region, product, sales amount).

FILES: **open lab-01-start.xlsm** · check against **lab-01-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON · LAB 2

Write Your First Macro in a Module

The sales manager wants a friendly confirmation whenever the commission tool is opened. You will insert a Module, write your first Sub procedure, and run it from the editor and from the Macro dialog.

YOU'LL BUILD

A Module1 containing a Welcome Sub procedure that greets the user with a message box.

✓ TEST IT

Running the Welcome macro from either the editor or the Macros dialog shows the message box 'Welcome to the NorthWind Commission Tracker'.

TOOLS: Visual Basic Editor, Module, Sub procedure, MsgBox

DATA: Uses NorthWind-Commission.xlsm from Lab 1.

FILES: **open lab-02-start.xlsm** · check against **lab-02-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON · LAB 3

Message Boxes and Button Choices

Before the tool recalculates commissions it must ask the user to confirm, because the operation overwrites last month's figures. You will use MsgBox with the vbYesNoCancel button set and read back which button was clicked.

YOU'LL BUILD

A ConfirmRecalculate Sub that asks the user to proceed and reports the button they chose.

✓ TEST IT

Each of the three buttons produces a different, correct follow-up message — Yes, No and Cancel are distinguished.

TOOLS: MsgBox, vbYesNoCancel, vbYes/vbNo/vbCancel constants

DATA: Uses NorthWind-Commission.xlsm from Lab 2.

FILES: **open lab-03-start.xlsm** · check against **lab-03-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON · LAB 4

Capture User Input with an Input Box

Commission rates change between quarters, so the tool must not hard-code them. You will use InputBox to ask the analyst for the rate, apply it to the selected sale, and display the result.

YOU'LL BUILD

A ShowCommission Sub that asks for a commission rate and reports the commission on the selected cell.

✓ TEST IT

Entering 0.1 with the $22,300 sale selected reports a commission of $2,230.00.

TOOLS: InputBox, ActiveCell, MsgBox

DATA: Uses the Sales sheet mock data from Lab 1.

FILES: **open lab-04-start.xlsm** · check against **lab-04-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON · LAB 5

Record a Macro to Calculate Sales Tax

The fastest way to discover Excel's object model is to let Excel write the code for you. You will record the steps for calculating 9% GST on each NorthWind sale, then open the recorded macro and read what the recorder generated.

YOU'LL BUILD

A recorded Cal_SalesTax macro plus your annotated reading of the generated code.

✓ TEST IT

Running Cal_SalesTax re-creates the GST formula in E2, giving 1116 for Aisha Rahman's $12,400 sale.

TOOLS: Macro Recorder, Developer ribbon, Module

DATA: Mock data: the January sales extended with a GST Rate column.

FILES: **open lab-05-start.xlsm** · check against **lab-05-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON · LAB 6

Run a Macro from a Button and a Shape

Sales staff will not open the VBA Editor. You will give them a one-click interface by placing a Form Control button and an assigned shape on the worksheet, wiring each to a macro — this is the 'simple Excel interface' element of LO1.

YOU'LL BUILD

A Sales sheet with a 'Calculate Commission' button and a shape that both launch macros.

✓ TEST IT

Clicking the 'Calculate Commission' button prompts for a rate, and clicking the shape shows the welcome message.

TOOLS: Developer → Insert, Form Controls, Assign Macro, Shapes

DATA: Uses NorthWind-Commission.xlsm from Lab 5.

FILES: **open lab-06-start.xlsm** · check against **lab-06-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Get Started on VBA

- You can now access the VBA development environment and save work in the correct macro-enabled format.
- You can now insert a module and write, save and run a first Sub procedure.
- You can now use MsgBox to display information and capture the user's button choice.
- You can now prompt the user for a value with InputBox and use it in a calculation.
- You can now use the macro recorder to generate VBA and then read the code it produces.
- You can now design a simple Excel interface by attaching macros to on-sheet controls.

© 2026 Tertiary Infotech Academy Pte Ltd

02

TOPIC 02

VBA Programming

Variables & constants · Data types & arrays · Operators · Decisions · Loops

© 2026 Tertiary Infotech Academy Pte Ltd

LO2 · CONCEPTS 1–5

Key Concepts — VBA Programming

1

Prompt-to-code specification

Tell the AI the exact workbook context, input cells, business rules, output and test case before asking for code.

2

Variables

A named memory location whose value can change while the macro runs — declare it with Dim before you use it.

3

Constants

A named value that must NOT change — declare with Const; attempting to reassign it raises an error.

4

Data types

Integer, Long, Double, Currency, String, Boolean, Date, Variant — choosing the right type prevents overflow and rounding bugs.

5

Option Explicit

Forces every variable to be declared, so a typo becomes a compile error instead of a silent empty value.

© 2026 Tertiary Infotech Academy Pte Ltd

LO2 · CONCEPTS 6–9

Key Concepts — VBA Programming

1

Arrays

One variable holding many values, indexed from 0 — ideal for reading a column of figures into memory at once.

2

Operators

Arithmetic (+ - * / ^), comparison (= <> > <), logical (And Or Not) and concatenation (&) build every expression.

3

Decisions

If…Then…ElseIf…Else runs different code per condition; Select Case is cleaner when testing one value against many bands.

4

Loops

For…Next counts, For Each walks a collection, Do While / Do Until repeat while a condition holds — Exit For / Exit Do break out early.

© 2026 Tertiary Infotech Academy Pte Ltd

Variables, Constants and Data Types

```
Option Explicit                      ' force every variable to be declared

Sub CalculateNetCommission()
    Const GST_RATE As Double = 0.09  ' cannot be changed at run time
    Dim strRep As String
    Dim curSales As Currency
    Dim curCommission As Currency

    curSales = ActiveCell.Value
    curCommission = curSales * 0.1
    MsgBox strRep & " earns " & Format(curCommission, "$#,##0.00")
End Sub
```

Dim declares a variable that may change · Const declares a value that must not · Option Explicit catches typos.

© 2026 Tertiary Infotech Academy Pte Ltd

Choosing a Data Type

1. **Integer**
   Whole numbers −32,768 to 32,767 — row counters and small counts.

2. **Long**
   Whole numbers beyond ±2 billion — safe for Excel row numbers.

3. **Double**
   Floating-point numbers — rates, percentages and scientific values.

4. **Currency**
   Fixed-point money with 4 decimal places — no rounding drift on cash.

5. **String**
   Text of any length — names, regions, product codes.

6. **Boolean**
   True or False — flags and conditions.

7. **Date**
   Dates and times — order dates, month ends.

8. **Variant**
   Anything at all — flexible, but slower and hides mistakes.

© 2026 Tertiary Infotech Academy Pte Ltd

BANDED COMMISSION RULES

Decisions — If…Then…ElseIf…Else

```
Sub ElseIfCommission()
    Dim curCommission As Currency

    If ActiveCell.Value > 10000 Then
        curCommission = Round(ActiveCell.Value * 0.08, 2)
    ElseIf ActiveCell.Value > 1000 Then
        curCommission = Round(ActiveCell.Value * 0.06, 2)
    Else
        curCommission = Round(ActiveCell.Value * 0.05, 2)
    End If

    MsgBox "Commission: " & curCommission
End Sub
```

Conditions are tested top to bottom — the first one that is True wins, and the rest are skipped.

© 2026 Tertiary Infotech Academy Pte Ltd

ONE VALUE, MANY BANDS

Select Case — Cleaner Than a Long Elself Chain

```
Sub SelectRate()
    Dim curTotal As Currency, curCommission As Currency
    curTotal = ActiveCell.Value

    Select Case curTotal
        Case Is >= 10000
            curCommission = curTotal * 0.08
        Case Is >= 1000
            curCommission = curTotal * 0.06
        Case Is >= 500
            curCommission = curTotal * 0.05
        Case Else
            curCommission = curTotal * 0.04
    End Select
End Sub
```

You will extend exactly this structure with a 7% band in the capstone lab.

© 2026 Tertiary Infotech Academy Pte Ltd

REPEATING WORK

Four Ways to Loop

1

For … Next

Repeat a known number of times: For i = 2 To 13 … Next. Add Step to change the increment.

2

For Each … Next

Walk every item in a collection or array — no counter needed.

3

Do While … Loop

Keep going WHILE a condition stays True — test at the top or the bottom.

4

Do Until … Loop

Keep going UNTIL a condition becomes True — ideal for finding the first empty row.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · 7 LABS

Hands-On Labs — VBA Programming

1

Lab 7: Declare Variables and Constants with the Right Data Type

A CalculateNetCommission Sub using Dim variables with correct types and a Const GST rate.

2

Lab 8: Build Expressions with VBA Operators

A SummariseRep Sub that computes figures and concatenates a one-line summary sentence.

3

Lab 9: Branch with If…Then…ElseIf…Else

An ElseIfCommission Sub that applies the correct banded rate to the selected sale.

4

Lab 10: Simplify Banded Logic with Select Case

A SelectRate Sub implementing four commission bands with Select Case.

5

Lab 11: Store a Column of Figures in an Array

A ShippingArray Sub that loads six monthly charges into an array and reports them.

6

Lab 12: Process Every Row with For…Next and For Each

A CommissionAllRows Sub and a ListRegions Sub demonstrating both loop styles.

7

Lab 13: Find the First Empty Row with Do…While and Do…Until

A FindFirstEmptyRow Sub that locates and reports the first blank row in the data.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON · LAB 7

Declare Variables and Constants with the Right Data Type

Hard-coded numbers scattered through a macro are impossible to maintain. You will rewrite the commission calculation using declared variables for the changing values and a Const for the GST rate that must never change, and see what happens when code tries to reassign a constant.

YOU'LL BUILD

A CalculateNetCommission Sub using Dim variables with correct types and a Const GST rate.

✓ TEST IT

The macro reports commission and GST for the selected rep, and assigning to GST_RATE raises 'Assignment to constant not permitted'.

TOOLS: Dim, Const, Option Explicit, Currency/Double/String/Integer types

DATA: Uses the Sales sheet mock data; commission rate 10%, GST fixed at 9%.

FILES: **open lab-07-start.xlsm** · check against **lab-07-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON · LAB 8

Build Expressions with VBA Operators

A commission engine is nothing but expressions. You will use all four operator families to compute a rep's gross commission, compare it against a bonus threshold, combine two conditions, and build a readable summary line for the manager's report.

YOU'LL BUILD

A SummariseRep Sub that computes figures and concatenates a one-line summary sentence.

✓ TEST IT

Aisha Rahman is reported as above target with a loyalty bonus; Hui Ling Ong is reported as below target.

TOOLS: Arithmetic (+ - * /), comparison (> < =), logical (And Or Not), concatenation (&)

DATA: Mock data: quarterly targets per rep so comparisons have something realistic to test against.

FILES: **open lab-08-start.xlsm** · check against **lab-08-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON · LAB 9

Branch with If…Then…ElseIf…Else

NorthWind pays banded commission: 8% above $10,000, 6% above $1,000, and 5% below that. You will implement the bands with If…Then…ElseIf…Else — the exact logic the capstone builds on.

YOU'LL BUILD

An ElseIfCommission Sub that applies the correct banded rate to the selected sale.

✓ TEST IT

$22,300 yields $1,784.00 at 8%, $5,300 yields $318.00 at 6%, and $980 yields $49.00 at 5%.

TOOLS: If…Then, ElseIf, Else, End If, Round

DATA: Uses the Sales sheet — the mock data deliberately spans all three commission bands.

FILES: **open lab-09-start.xlsm** · check against **lab-09-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON · LAB 10

Simplify Banded Logic with Select Case

The finance team adds a fourth commission band, and the ElseIf chain is getting hard to read. You will rewrite the same logic with Select Case — the structure the capstone asks you to extend.

YOU'LL BUILD

A SelectRate Sub implementing four commission bands with Select Case.

✓ TEST IT

Devi Kumar's $15,900 returns $1,272.00 at 8%, and Hui Ling Ong's $640 returns $32.00 at 5%.

TOOLS: Select Case, Case Is, Case Else

DATA: Bands: 8% ≥ $10,000 · 6% ≥ $1,000 · 5% ≥ $500 · 4% below $500.

FILES: **open lab-10-start.xlsm** · check against **lab-10-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON · LAB 11

Store a Column of Figures in an Array

Reading the same cells repeatedly is slow and fragile. You will load NorthWind's six monthly shipping charges into an array, then read them back by index — the foundation for loop processing.

YOU'LL BUILD

A ShippingArray Sub that loads six monthly charges into an array and reports them.

✓ TEST IT

The message box lists all six charges beginning with 320.5 and ending with 447.9.

TOOLS: Dim with parentheses, array index, Offset

DATA: Mock data: six months of shipping charges for the Central region.

FILES: **open lab-11-start.xlsm** · check against **lab-11-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON · LAB 12

Process Every Row with For…Next and For Each

Twelve reps today, two hundred next year — the tool must scale. You will calculate commission for every row with a For…Next loop, then use For Each to walk a list of sales regions, and reverse the loop's direction to see how Step works.

YOU'LL BUILD

A CommissionAllRows Sub and a ListRegions Sub demonstrating both loop styles.

✓ TEST IT

All 12 commission cells populate in one run, and ListRegions stops after reporting North.

TOOLS: For…Next, Step, For Each…Next, Exit For, Cells

DATA: Uses the Commission sheet from Lab 10 (12 rows) plus a regions array.

FILES: **open lab-12-start.xlsm** · check against **lab-12-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON · LAB 13

Find the First Empty Row with Do…While and Do…Until

New sales records must be appended below the existing ones, so the macro has to find the first empty row. You will solve this with a Do loop — the same technique your UserForm will use in Topic 5 to append records.

YOU'LL BUILD

A FindFirstEmptyRow Sub that locates and reports the first blank row in the data.

✓ TEST IT

Both the Do Until and Do While versions report 'First empty row: 14'.

TOOLS: Do While, Do Until, Loop, Exit Do, Offset, Activate

DATA: Uses the Sales sheet; the mock data ends at row 13 so row 14 is the expected result.

FILES: **open lab-13-start.xlsm** · check against **lab-13-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — VBA Programming

- You can now declare variables and constants using appropriate data types.
- You can now apply arithmetic, comparison, logical and concatenation operators.
- You can now control execution flow with conditional statements.
- You can now use Select Case as a clearer alternative to long ElseIf chains.
- You can now declare an array and assign values to it by index.
- You can now repeat operations over a range using counted and collection loops.
- You can now repeat operations while or until a condition is met.

© 2026 Tertiary Infotech Academy Pte Ltd

03

TOPIC 03

Function and Sub Procedure

Function vs Sub · Arguments & return values · Scope · Worksheet functions · Charts

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Function and Sub Procedure

1

AI-generated procedures

Ask for one small procedure with an explicit contract, then verify its arguments, return value, scope and side effects.

2

Sub procedure

A block of statements between Sub and End Sub that performs an action but returns no value — this is what a macro button runs.

3

Function procedure

Sits between Function and End Function and RETURNS a value by assigning to its own name — usable from a worksheet cell.

4

The key difference

A Function may return a value; a Sub does not — know this distinction precisely.

5

Arguments

Values passed into a procedure, e.g. Cal_Commission(curSales) — they make one procedure reusable for many inputs.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Function and Sub Procedure

1

Scope

Public procedures are visible to the whole project; Private ones only to their own module — control what other code can call.

2

Modular design

Break a long macro into small, named procedures that each do one thing; call them from a short master Sub.

3

Worksheet functions

Application.WorksheetFunction lets VBA reuse Excel's built-ins — Sum, Min, Max, Pmt — instead of re-implementing them.

4

Charts from code

ActiveSheet.Shapes.AddChart with ChartType and SetSourceData builds a chart programmatically from a data range.

© 2026 Tertiary Infotech Academy Pte Ltd

KEY DISTINCTION

Function vs Sub Procedure

Function

- Returns a value to whatever called it.
- Assigns the result to its own name.
- Can be used in a worksheet cell like =Cal_Commission(C2).
- Wrapped in Function … End Function.
- Use when you need an answer back.

Sub Procedure

- Performs an action; returns no value.
- Cannot be used in a worksheet cell.
- Run from a button, shape or the Macros dialog.
- Wrapped in Sub … End Sub.
- Use when you need something done.

© 2026 Tertiary Infotech Academy Pte Ltd

WRITE ONCE, CALL ANYWHERE

A Reusable User-Defined Function

```
Function Cal_Commission(curSales As Currency) As Currency
    ' Returns 10% commission on the sales figure supplied
    Cal_Commission = curSales * 0.1
End Function

' In VBA:      curPay = Cal_Commission(12400)
' In a cell:   =Cal_Commission(C2)
```

A Function returns its value by assigning to its own name — this one works in code AND in a worksheet cell.

© 2026 Tertiary Infotech Academy Pte Ltd

CONTROLLING VISIBILITY

Scope — Who Can Call What

1

Public

Visible to every module in the project and listed in the Macros dialog — your tool's front door.

2

Private

Visible only inside its own module and hidden from the Macros dialog — internal helpers.

3

Why it matters

Users should only see the procedures meant for them, not every internal step.

4

Default

A procedure with no keyword is Public — state it explicitly so intent is clear.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · 5 LABS

Hands-On Labs — Function and Sub Procedure

1

Lab 14: Write a Reusable Function that Returns a Value

A Cal_Commission function callable from both VBA and any worksheet cell.

2

Lab 15: Compare a Function with a Sub Procedure

A matched pair, Cal_Commission (Function) and ShowCommissionSub (Sub), plus a written comparison.

3

Lab 16: Control Visibility with Public and Private Scope

A module where internal helpers are Private and only the master macro is Public.

4

Lab 17: Reuse Excel's Built-in Worksheet Functions

A SalesStatistics Sub reporting total, average, highest and lowest sales, plus a PMT calculation.

5

Lab 18: Generate a Commission Chart from Code

A PlotCommission macro that builds a bar chart, wired to a 'Plot Commission' button.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON · LAB 14

Write a Reusable Function that Returns a Value

The 10% commission calculation is now duplicated in three macros. You will extract it into a single user-defined Function that takes a sales figure and returns the commission — then call it from a worksheet cell exactly like SUM or AVERAGE.

YOU'LL BUILD

A Cal_Commission function callable from both VBA and any worksheet cell.

✓ TEST IT

=Cal_Commission(C2) in a worksheet cell returns 1240 for Aisha Rahman's $12,400 sale.

TOOLS: Function…End Function, arguments, return value, UDF in a cell

DATA: Uses the Commission sheet mock data from Topic 2.

FILES: **open lab-14-start.xlsm** · check against **lab-14-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON · LAB 15

Compare a Function with a Sub Procedure

You will build the same commission logic twice — once as a Function that returns a value, once as a Sub that performs an action — and record precisely how they differ in use.

YOU'LL BUILD

A matched pair, Cal_Commission (Function) and ShowCommissionSub (Sub), plus a written comparison.

✓ TEST IT

The Function works in a worksheet cell and returns a value; the Sub does not work in a cell but runs from the Macros dialog.

TOOLS: Function vs Sub, Call keyword, return values

DATA: Uses the Commission sheet; no new data required.

FILES: **open lab-15-start.xlsm** · check against **lab-15-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON · LAB 16

Control Visibility with Public and Private Scope

The commission workbook now has helper procedures that should not appear in the user's Macros list. You will use Private and Public to control exactly what is exposed.

YOU'LL BUILD

A module where internal helpers are Private and only the master macro is Public.

✓ TEST IT

Only the Public procedures are listed in the Macros dialog, and Module2 cannot call the Private helper.

TOOLS: Public, Private, module scope, Macros dialog

DATA: Uses NorthWind-Commission.xlsm as built so far.

FILES: **open lab-16-start.xlsm** · check against **lab-16-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON · LAB 17

Reuse Excel's Built-in Worksheet Functions

Do not re-implement what Excel already does. You will use Application.WorksheetFunction to total and summarise NorthWind's sales, and to compute a loan repayment with PMT for the company's new vehicle financing.

YOU'LL BUILD

A SalesStatistics Sub reporting total, average, highest and lowest sales, plus a PMT calculation.

✓ TEST IT

Total January sales report as $92,470, and the monthly loan repayment is approximately $1,549 — matching Excel's own =PMT result.

TOOLS: Application.WorksheetFunction (Sum, Average, Max, Min, Pmt), Range

DATA: Mock data: vehicle loan terms — $18,000 over 12 months at 6% annual interest.

FILES: **open lab-17-start.xlsm** · check against **lab-17-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON · LAB 18

Generate a Commission Chart from Code

The sales manager wants the commission breakdown as a chart, refreshed on demand. You will write a macro that creates a bar chart from the commission range and attach it to a button — the same capability the capstone requires.

YOU'LL BUILD

A PlotCommission macro that builds a bar chart, wired to a 'Plot Commission' button.

✓ TEST IT

Clicking the 'Plot Commission' button draws a clustered column chart of commission by rep.

TOOLS: Shapes.AddChart, ChartType, xlColumnClustered, SetSourceData, Form Control button

DATA: Charts the rep names and commission values on the Commission sheet.

FILES: **open lab-18-start.xlsm** · check against **lab-18-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Function and Sub Procedure

- You can now create a user-defined function that accepts arguments and returns a result.
- You can now distinguish a Function from a Sub procedure and choose the right one.
- You can now apply Public and Private scope to control where procedures can be called.
- You can now call Excel's built-in functions from VBA with Application.WorksheetFunction.
- You can now produce a chart programmatically to visualise results.

© 2026 Tertiary Infotech Academy Pte Ltd

04

TOPIC 04

Managing Excel Objects

Application, Workbook, Worksheet & Range objects · Errors · Error handling · Testing & debugging

© 2026 Tertiary Infotech Academy Pte Ltd

LO4 · CONCEPTS 1–5

Key Concepts — Managing Excel Objects

1

Review before run

Inspect AI-generated object references, file operations and error paths; test in a copy before trusting the macro.

2

The object model

Excel exposes a hierarchy — Application → Workbook → Worksheet → Range — and VBA drives every level of it.

3

Range object

A cell, row, column or block; read and write .Value, move with .Offset, select regions and copy or paste them.

4

Workbook & Worksheet

Open, add, activate, protect and close files and sheets from code — the backbone of multi-file automation.

5

Three kinds of error

Syntax errors break compilation, runtime errors stop execution mid-run, and logical errors run fine but give a wrong answer.

© 2026 Tertiary Infotech Academy Pte Ltd

LO4 · CONCEPTS 6–9

Key Concepts — Managing Excel Objects

1

Error handling

On Error GoTo <label> diverts to a handler; Err.Number identifies what failed so the macro can respond and continue.

2

Why handle errors

Unhandled runtime errors dump the user into the code window — a handled error shows a clear message instead.

3

Debugging tools

Breakpoints (F9) pause execution, Step Into (F8) walks line by line, and the Watch window inspects values as they change.

4

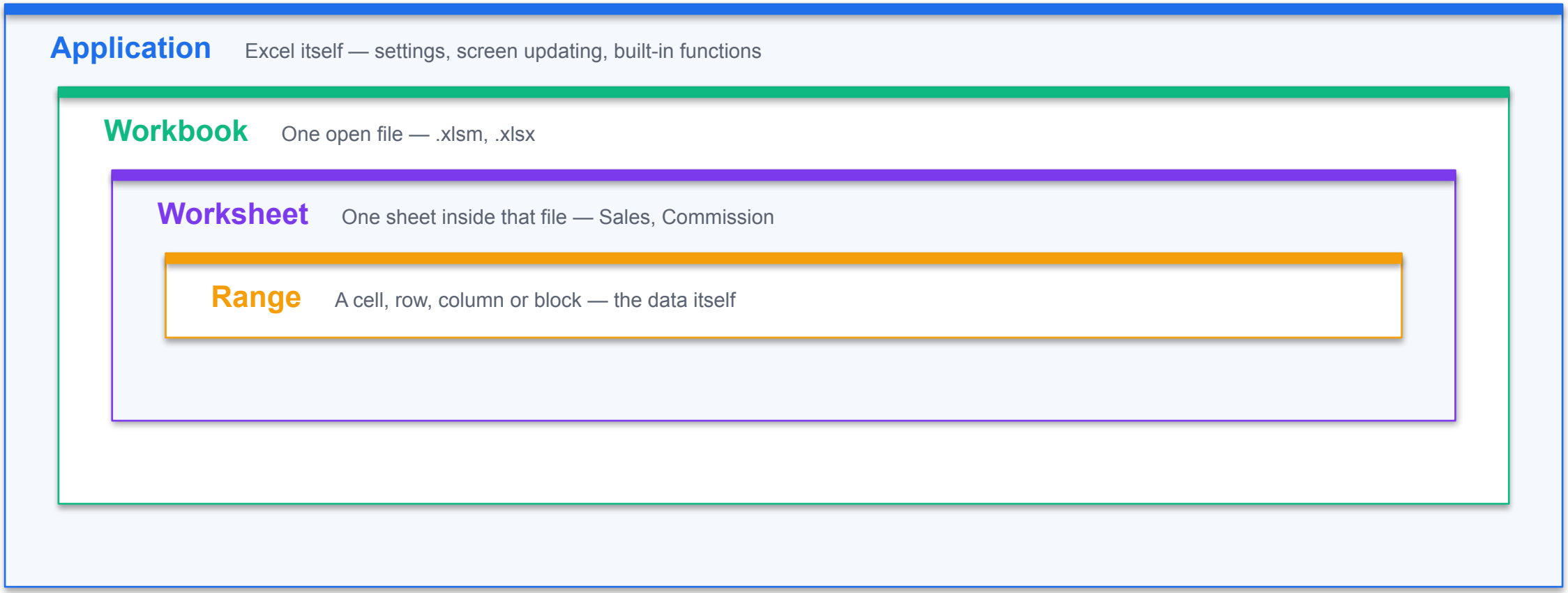
Testing

Run each procedure against known inputs and confirm the expected output before trusting it with production data.

© 2026 Tertiary Infotech Academy Pte Ltd

EVERYTHING NESTS

The Excel Object Model



Worksheets("Sales").Range("E2").Value — read the path from the outside in.

© 2026 Tertiary Infotech Academy Pte Ltd

READ, WRITE, MOVE

Working with the Range Object

```
' Read a value
curSales = Worksheets("Sales").Range("E2").Value

' Write a value
Worksheets("Sales").Range("F2").Value = curSales * 0.1

' Move relative to the current cell
ActiveCell.Offset(1, 0).Activate       ' down one row

' Address a cell by row and column number
Cells(2, 6).Value = "Commission"
```

Range("A1") uses the address you see; Cells(row, col) uses numbers — ideal inside loops.

© 2026 Tertiary Infotech Academy Pte Ltd

KNOW THE DIFFERENCE

Three Kinds of Error

1

Syntax error

Broken grammar — a missing bracket or keyword. Caught when you compile; the code will not run at all.

2

Runtime error

Valid code that fails while running — dividing by zero, or a sheet that does not exist. Stops execution mid-macro.

3

Logical error

Runs perfectly and reports nothing wrong, but the answer is incorrect — a rate of 0.01 instead of 0.1.

4

Why it matters

Only the logical error survives to production, which is why every procedure must be tested against known values.

© 2026 Tertiary Infotech Academy Pte Ltd

KEY PATTERN

Error Handling — On Error GoTo

```
Sub CheckError()
    Dim a, b As Integer
    Dim c As Double

    On Error GoTo ErrorHandler:
    a = InputBox("Enter the first number.")
    b = InputBox("Enter the second number.")
    c = a / b
    MsgBox c
    Exit Sub

ErrorHandler:
    Select Case Err.Number
        Case 11    ' Divide by zero error
            MsgBox "You attempted to divide by zero!"
    End Select
End Sub
```

On Error GoTo <label> diverts to the handler · Err.Number identifies the failure · Exit Sub keeps success out of the handler.

© 2026 Tertiary Infotech Academy Pte Ltd

Debugging Toolkit

1

Breakpoint (F9)

Click the margin to set a red dot — execution pauses there so you can inspect state.

2

Step Into (F8)

Walk the macro one line at a time and watch exactly which branch runs.

3

Watch window

Track a variable's value continuously as the loop iterates.

4

Immediate window (Ctrl+G)

Debug.Print writes values here — the fastest way to see what your code is really doing.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · 7 LABS

Hands-On Labs — Managing Excel Objects

1

Lab 19: Read and Write Cells with the Range Object

A FormatSalesReport Sub that renames headings and writes calculated values into the sheet.

2

Lab 20: Copy Data Between Worksheets

A CopyToSummary Sub that consolidates the January data onto a Summary sheet.

3

Lab 21: Automate Workbooks and Worksheets

A MonthEndSetup Sub that adds and names a sheet, and reports on open workbooks.

4

Lab 22: Identify Syntax, Runtime and Logical Errors

Three annotated broken procedures demonstrating each error type and its symptom.

5

Lab 23: Handle Runtime Errors with On Error GoTo

A CheckError Sub with a labelled error handler that traps division-by-zero and bad input.

6

Lab 24: Debug with Breakpoints, Step Into and the Watch Window

A debugging session on the commission loop with a breakpoint and a watch expression.

7

Lab 25: Find a Value and Handle the Not-Found Case

A FindThisValue Sub that locates a rep and reports clearly when the name is not found.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON · LAB 19

Read and Write Cells with the Range Object

Everything a macro does to data goes through the Range object. You will read values, write values, rename a column heading and move around the sheet with Offset — the core skills for automating NorthWind's monthly report.

YOU'LL BUILD

A FormatSalesReport Sub that renames headings and writes calculated values into the sheet.

✓ TEST IT

Cell F1 reads 'Commission' and F2 holds the calculated commission for the first rep.

TOOLS: Range, .Value, .Offset, .Activate, Cells

DATA: Uses the Sales sheet mock data from Lab 1.

FILES: **open lab-19-start.xlsm** · check against **lab-19-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON · LAB 20

Copy Data Between Worksheets

NorthWind's finance team needs January's figures consolidated onto a Summary sheet. You will automate the copy-and-paste, then rewrite it as a direct value assignment and compare the two.

YOU'LL BUILD

A CopyToSummary Sub that consolidates the January data onto a Summary sheet.

✓ TEST IT

The Summary sheet holds a complete copy of the January data, and no clipboard border remains on the Sales sheet.

TOOLS: Worksheets.Activate, Range.Select, Selection.Copy, ActiveSheet.Paste, Value assignment

DATA: Uses the Sales sheet as the source; creates a Summary sheet as the destination.

FILES: **open lab-20-start.xlsm** · check against **lab-20-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON · LAB 21

Automate Workbooks and Worksheets

Month-end means creating a fresh sheet, naming it and checking whether last month's file is already open. You will drive the Workbook and Worksheet objects to do all three.

YOU'LL BUILD

A MonthEndSetup Sub that adds and names a sheet, and reports on open workbooks.

✓ TEST IT

A February sheet is created and named, the sheet count is reported correctly, and hiding then showing the sheet works.

TOOLS: Workbooks.Add/Open/Close, Worksheets.Add, .Name, .Activate, .Visible

DATA: Uses NorthWind-Commission.xlsm as built so far.

FILES: **open lab-21-start.xlsm** · check against **lab-21-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON · LAB 22

Identify Syntax, Runtime and Logical Errors

Before you can handle errors you must recognise them. You will deliberately introduce one of each kind into the commission macro and observe how differently each one fails — the logical error is the dangerous one, because Excel reports no problem at all.

YOU'LL BUILD

Three annotated broken procedures demonstrating each error type and its symptom.

✓ TEST IT

You can state which error is caught at compile time, which stops execution mid-run, and which produces a silently wrong answer.

TOOLS: VBA compiler, runtime error dialog, Debug

DATA: Uses the Commission sheet mock data.

FILES: **open lab-22-start.xlsm** · check against **lab-22-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON · LAB 23

Handle Runtime Errors with On Error GoTo

An analyst typing 'ten' instead of 10 must not be dumped into the code window. You will add an error handler that catches the failure, identifies it via Err.Number and shows a helpful message.

YOU'LL BUILD

A CheckError Sub with a labelled error handler that traps division-by-zero and bad input.

✓ TEST IT

Dividing by zero shows 'You attempted to divide by zero!' instead of Excel's runtime error dialog, and the macro ends cleanly.

TOOLS: On Error GoTo, error label, Err.Number, Select Case, Exit Sub

DATA: Prompts the user for two numbers; test with 10 and 0, and with 10 and 'ten'.

FILES: **open lab-23-start.xlsm** · check against **lab-23-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON · LAB 24

Debug with Breakpoints, Step Into and the Watch Window

A commission figure is wrong and you cannot see why by reading the code. You will pause the macro on a breakpoint, walk it one line at a time, and watch a variable change value as the loop runs.

YOU'LL BUILD

A debugging session on the commission loop with a breakpoint and a watch expression.

✓ TEST IT

Execution pauses at the breakpoint, F8 advances one line per press, and the Immediate window lists a commission line per row.

TOOLS: Breakpoints (F9), Step Into (F8), Watch window, Immediate window, Debug.Print

DATA: Uses the CommissionAllRows loop from Lab 12.

FILES: **open lab-24-start.xlsm** · check against **lab-24-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 04 · HANDS-ON · LAB 25

Find a Value and Handle the Not-Found Case

The manager wants to look up any rep's record by name. Cells.Find raises a runtime error when there is no match, so this lab combines the object model with the error handling from Lab 23.

YOU'LL BUILD

A FindThisValue Sub that locates a rep and reports clearly when the name is not found.

✓ TEST IT

Searching 'Devi Kumar' selects that cell and reports its address; searching 'John Smith' reports 'Value not found.' with no crash.

TOOLS: Cells.Find, xlWhole, On Error GoTo, Activate

DATA: Search the Sales sheet for 'Devi Kumar' (present) and 'John Smith' (absent).

FILES: **open lab-25-start.xlsm** · check against **lab-25-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Managing Excel Objects

- You can now manage worksheet data through the Range object.
- You can now manage interoperability by moving data across worksheets.
- You can now manage workbook and worksheet objects from code.
- You can now identify the three types of programming error.
- You can now identify relevant controls by trapping and responding to runtime errors.
- You can now test and debug code using the VBA Editor's debugging tools.
- You can now combine object handling with error control in a realistic lookup.

© 2026 Tertiary Infotech Academy Pte Ltd

05

TOPIC 05

Create User Defined Forms

Workbook & worksheet events · UserForms & controls · Event procedures · Documentation

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Create User Defined Forms

1

Prompt-to-code evidence

Document the prompt, generated code, learner edits and acceptance tests so the automation is explainable and maintainable.

2

Event-driven programming

VBA code can run automatically in response to something happening, rather than only when a user clicks a button.

3

Workbook events

Workbook_Open, Workbook_BeforeClose and Workbook_BeforeSave fire for the file as a whole — set Cancel=True to block the action.

4

Worksheet events

Worksheet_Change and Worksheet_BeforeDoubleClick fire for one sheet; add them via right-click → View Code on the sheet tab.

5

UserForms

A custom dialog box that makes data entry controlled and user-friendly instead of typing straight into cells.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Create User Defined Forms

1

Form controls

Labels, TextBoxes, ComboBoxes, OptionButtons, ListBoxes and CommandButtons — each has a Name used in code and a Caption seen by the user.

2

Event procedures

UserForm_Initialize prepares the form as it opens; CommandButton_Click writes the captured values to the worksheet.

3

Validating input

Check the form's fields before writing them so bad or blank data never reaches the sheet.

4

Documenting the solution

Comments (') explain intent, meaningful names explain structure, and a short user note explains how to run the tool.

© 2026 Tertiary Infotech Academy Pte Ltd

CODE THAT RUNS BY ITSELF

Event-Driven Programming

1

Workbook_Open

Fires when the file is opened — greet the user or prepare the sheets.

2

Workbook_BeforeClose

Fires as the file closes — set Cancel = True to block the close until data is complete.

3

Workbook_BeforeSave

Fires before saving — validate or stamp the file.

4

Worksheet_Change

Fires when a cell on that sheet is edited — confirm or reject the edit.

5

Worksheet_BeforeDoubleClick

Fires on a double-click — trigger a lookup or a custom action.

6

Where they live

Workbook events go in ThisWorkbook; worksheet events go in that sheet's own code window.

© 2026 Tertiary Infotech Academy Pte Ltd

KEY PATTERN

A Workbook Event Procedure

```
Private Sub Workbook_BeforeClose(Cancel As Boolean)
    ' Block the close until the month's commission rate is entered
    If Worksheets("Commission").Range("F5").Value = "" Then
        MsgBox "Please enter a commission rate in F5 before closing."
        Cancel = True
    End If
End Sub
```

Setting Cancel = True inside a Before… event stops the action Excel was about to take.

© 2026 Tertiary Infotech Academy Pte Ltd

UserForm Controls

1

Label

Static caption text that tells the user what a field is for.

2

TextBox

Free typing — names, amounts. Named txtRepName, txtSalesAmount.

3

ComboBox

A dropdown list of allowed values — stops invalid regions being typed.

4

OptionButton

Mutually exclusive choices — gender, payment type.

5

ListBox

A visible list allowing single or multiple selection.

6

CommandButton

Runs code when clicked — cmdSubmit, cmdCancel.

© 2026 Tertiary Infotech Academy Pte Ltd

VALIDATE, THEN APPEND

Writing the Form's Data to the Sheet

```
Private Sub cmdSubmit_Click()
    Dim lngRow As Long

    If Trim(txtRepName.Value) = "" Then
        MsgBox "Please enter the rep name."
        txtRepName.SetFocus
        Exit Sub
    End If

    lngRow = Worksheets("Sales").Range("A1048576").End(xlUp).Row + 1
    Cells(lngRow, 1) = txtRepID.Value
    Cells(lngRow, 2) = txtRepName.Value
    Cells(lngRow, 3) = cboRegion.Value
    Cells(lngRow, 5) = txtSalesAmount.Value
End Sub
```

Validate every field first, find the first empty row, then write — never write unchecked data.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · 6 LABS

Hands-On Labs — Create User Defined Forms

1

Lab 26: Respond to Workbook Events

ThisWorkbook event procedures that run on open and guard the close.

2

Lab 27: Respond to Worksheet Events

A Worksheet_Change event on the Commission sheet that confirms every manual edit.

3

Lab 28: Design a UserForm for Sales Entry

A frmSalesEntry UserForm laid out with named, captioned controls.

4

Lab 29: Write the Form's Event Procedures

A working frmSalesEntry that appends each submitted record to the Sales sheet.

5

Lab 30: Validate Form Input and Document the Solution

A validated frmSalesEntry plus an in-workbook ReadMe documenting the solution.

6

Lab 31: Capstone — Complete the NorthWind Commission Tracker

The completed NorthWind-Commission.xlsm automating capture, calculation, validation and reporting.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON · LAB 26

Respond to Workbook Events

Analysts keep closing the commission workbook before entering the month's rate, leaving the report blank. You will use Workbook_Open and Workbook_BeforeClose to greet the user on open and block closing until the rate is filled in.

YOU'LL BUILD

ThisWorkbook event procedures that run on open and guard the close.

✓ TEST IT

Opening the file greets the user, and closing with F5 empty is blocked with a prompt until a rate is entered.

TOOLS: ThisWorkbook, Workbook_Open, Workbook_BeforeClose, Cancel argument

DATA: Uses the Commission sheet; cell F5 holds the month's commission rate.

FILES: **open lab-26-start.xlsm** · check against **lab-26-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON · LAB 27

Respond to Worksheet Events

Commission figures are generated, not typed — an analyst overtyping one silently corrupts the report. You will use Worksheet_Change to ask for confirmation whenever a cell on the Commission sheet is edited.

YOU'LL BUILD

A Worksheet_Change event on the Commission sheet that confirms every manual edit.

✓ TEST IT

Editing any cell on the Commission sheet prompts for confirmation, and choosing Cancel restores the previous value.

TOOLS: Worksheet_Change, Target argument, EnableEvents, MsgBox vbOKCancel

DATA: Uses the Commission sheet mock data.

FILES: **open lab-27-start.xlsm** · check against **lab-27-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON · LAB 28

Design a UserForm for Sales Entry

Typing straight into cells causes bad data. You will design a proper data-entry dialog with labels, text boxes, a combo box for region and Submit/Cancel buttons, naming every control so the code that follows is readable.

YOU'LL BUILD

A frmSalesEntry UserForm laid out with named, captioned controls.

✓ TEST IT

Running frmSalesEntry shows a tidy dialog with four labelled inputs, a region dropdown and two buttons.

TOOLS: Insert → UserForm, Toolbox, Label, TextBox, ComboBox, CommandButton, Properties window

DATA: Captures RepID, Rep Name, Region and Sales Amount to match the Sales sheet columns.

FILES: **open lab-28-start.xlsm** · check against **lab-28-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON · LAB 29

Write the Form's Event Procedures

A form that does nothing is decoration. You will fill the region list as the form opens, append the captured record to the first empty row on submit — reusing the Do loop from Lab 13 — and close the form on cancel.

YOU'LL BUILD

A working frmSalesEntry that appends each submitted record to the Sales sheet.

✓ TEST IT

Submitting three records appends three new rows to the Sales sheet below the existing data, and Cancel closes the form.

TOOLS: UserForm_Initialize, CommandButton_Click, AddItem, End(xlUp).Row, Unload Me

DATA: Regions loaded into the combo box: Central, East, North, West.

FILES: **open lab-29-start.xlsm** · check against **lab-29-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON · LAB 30

Validate Form Input and Document the Solution

A blank name or a text sales figure must never reach the sheet. You will validate every field before writing, and then document the finished tool so a colleague can run and maintain it.

YOU'LL BUILD

A validated frmSalesEntry plus an in-workbook ReadMe documenting the solution.

✓ TEST IT

Blank names, non-numeric amounts and missing regions are each rejected with a clear message and no row is written.

TOOLS: IsNumeric, Trim, SetFocus, comments, documentation sheet

DATA: Test cases: blank name, non-numeric sales amount, and a valid record.

FILES: **open lab-30-start.xlsm** · check against **lab-30-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 05 · HANDS-ON · LAB 31

Capstone — Complete the NorthWind Commission Tracker

Bring every topic together into the finished deliverable: banded commission with Select Case, a launch button, the validated entry form, error handling and an on-demand chart.

YOU'LL BUILD

The completed NorthWind-Commission.xlsm automating capture, calculation, validation and reporting.

✓ TEST IT

From a fresh open: the welcome fires, 'New Sale' captures a validated record, commission is banded correctly including the 7% band, 'Plot Commission' charts the result, and the file reopens with everything intact.

TOOLS: Select Case, UserForm, error handling, charts, Form Controls, .xlsm

DATA: Final bands: 8% ≥ $10,000 · 7% ≥ $5,000 · 6% ≥ $1,000 · 5% ≥ $500 · 4% below $500.

FILES: **open lab-31-start.xlsm** · check against **lab-31-end.xlsm** · AI prompt, review checklist and full steps are in the Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Create User Defined Forms

- You can now write event procedures that run automatically at workbook level.
- You can now write event procedures that react to changes on a worksheet.
- You can now produce a user-defined form with appropriately named controls.
- You can now produce event procedures that initialise a form and write its data to the worksheet.
- You can now identify relevant controls for data quality and produce solution documentation.
- You can now integrate the full course into one working automated solution.

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES

What You Achieved

1

LO1 Designed an interface

Enabled the Developer tools, wrote macros in modules and drove them from buttons and shapes.

2

LO2 Programmed in VBA

Declared typed variables and constants, built expressions, branched with If and Select Case, and looped.

3

LO3 Built procedures

Wrote reusable functions and subs, controlled scope, reused worksheet functions and generated charts.

4

LO4 Managed Excel objects

Drove Workbook, Worksheet and Range objects, handled runtime errors and debugged with breakpoints.

5

LO5 Produced forms

Built a validated UserForm with event procedures and documented the finished solution.

© 2026 Tertiary Infotech Academy Pte Ltd

THE CAPSTONE

You built a real automation.

The NorthWind Commission Tracker captures a sale on a validated form, applies banded commission, handles bad input and charts the result.

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

Keep Going After Today

1

Automate one real task

Pick the most repetitive job in your own workbooks and rebuild it as a macro this week.

2

Read the recorder's code

Record anything you do not know how to code, then read what Excel wrote.

3

Keep a prompt-and-test log

Retain the prompt, generated draft, learner edits and acceptance evidence for every automation.

4

Build a reviewed library

Keep only tested, documented procedures in a module you can import into a new workbook.

© 2026 Tertiary Infotech Academy Pte Ltd

CONTINUE YOUR LEARNING

Recommended Courses

- Data Analytics and Visualisation with Power BI
- Statistical Data Analysis with Excel
- Create Interactive Dashboard Using Excel
- Data Visualisation with Tableau
- Browse the full catalogue at www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd

WE'RE HERE TO HELP · DURING AND AFTER THE CLASS

Support

1

Email

enquiry@tertiaryinfotech.com

2

Telephone

+65 6100 0613

3

Website

www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE YOU LEAVE

Course Feedback

1

Share your feedback

Complete the short course feedback form on the LMS before you leave.

2

Tell us what worked

Which labs, prompts and demos helped most — and what should we improve?

3

Keep your evidence

Save your prompts, final VBA and test results as your reference after class.

4

Certificate of completion

Your certificate of completion will be emailed to you after the course.

© 2026 Tertiary Infotech Academy Pte Ltd

HAPPY VIBE CODING

Thank You!

You can now prompt, review, test, debug and document an AI-assisted Excel VBA automation.

© 2026 Tertiary Infotech Academy Pte Ltd